

32 KiB 4-way set-associative cache

1. Requirements and parameters

Design and simulate a **32 KiB, 4-way set-associative cache**, with 8-word blocks and 32-bit words. Main memory is 8 MiB, the addressable unit is the word, and the cache is write-back with write-allocate and an LRU replacement policy.

Cache geometry

32 KiB = 2^{15} B. With 32 bits = 4 B = 2^2 B per word, the cache holds $2^{15} / 2^2 = 2^{13}$ words. With 8 words per block (2^3), that gives $2^{13} / 2^3 = 2^{10} = 1024$ blocks. Because the cache is 4-way set-associative, the 1024 blocks are grouped into $1024 / 4 = 256 = 2^8$ sets, so the index field is 8 bits.

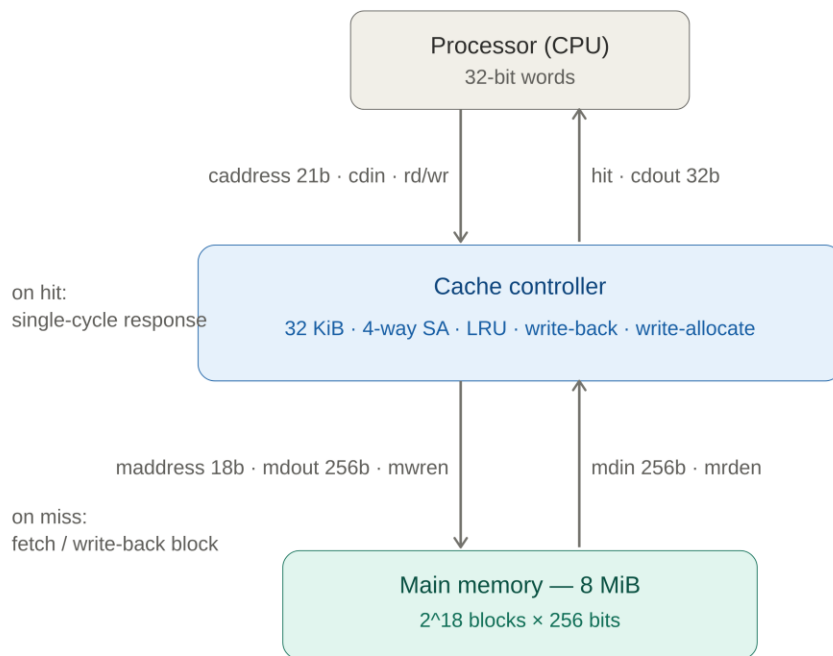
Main memory

8 MiB = 2^{23} B; with 4-byte words there are 2^{21} words, i.e. a 21-bit word address. The tag is therefore $21 - 8 - 3 = 10$ bits. The address issued to memory is {tag, index} = 18 bits, since memory is addressed by block.

Tag (20:11)	Index (10:3)	Block offset (2:0)
10 bits	8 bits - 256 sets	3 bits - 8 words

Parameter	Value
BLOCK_SIZE	256 bits (8×32)
ADDRESS_WIDTH	21 bits
INDEX_WIDTH	8 bits (256 sets)
TAG_WIDTH	10 bits
OFFSET_WIDTH	3 bits
WORD_SIZE	32 bits
NWAYS	4
NSETS	256

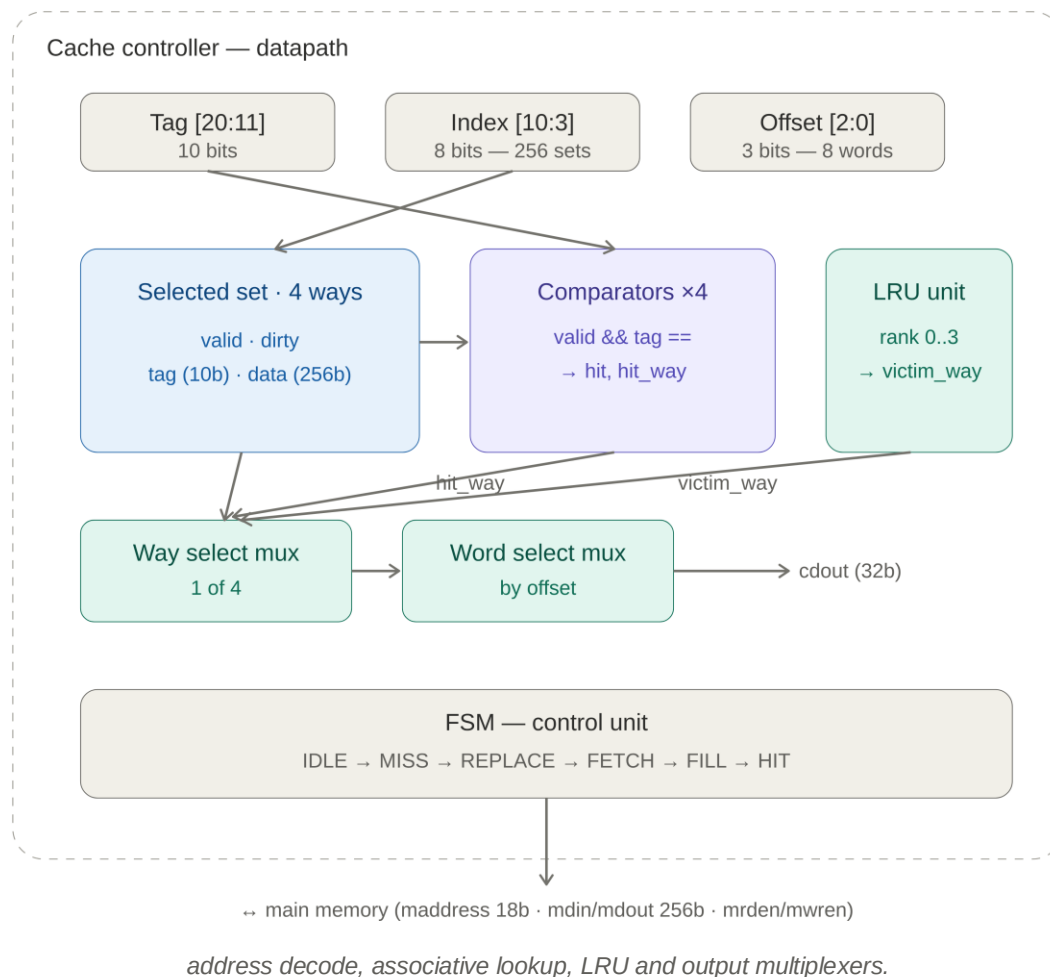
2. System block diagram



CPU <-> cache controller <-> main memory.

On the bus between the CPU and the cache a single 32-bit word travels together with the address; on the bus between the cache and memory **full 256-bit blocks** (8 words) travel - either fetched on a miss (mdin) or written back when a modified (dirty) line is evicted (mdout). The address to memory is {tag, index} = 18 bits, because memory is addressed by block.

3. Internal datapath of the controller



Reading the datapath, step by step:

- **Address decode** - the 21 bits are split into tag (10b), index (8b) and offset (3b). The index selects one of the 256 sets, the tag goes to the comparators, the offset to the word multiplexer.
- **Selected set** - for the chosen set all 4 ways are read out (valid, dirty, tag, data). This is the cache's SRAM storage.
- **Comparators ×4** - each way compares its stored tag with the requested tag and checks the valid bit. If any matches, **hit = 1** and **hit_way** is encoded (which way hit). This is the essence of set-associativity: the lookup happens in parallel across all ways.
- **LRU unit** - keeps a rank 0..3 for each way in the set and produces **victim_way** (the way to replace on a miss: first an invalid way, otherwise the one with rank 0).
- **Way mux -> word mux** - the way mux picks the correct block (**hit_way** on a hit, **victim_way** on a fill), and the word mux extracts the 32-bit word by offset → **cdout**.

- **FSM** - controls everything: on a hit it responds in one cycle; on a miss it goes MISS -> (REPLACE, if the victim line is dirty -> write-back) -> FETCH -> FILL -> HIT, driving `mrden/mwren/maddress` toward memory.

4. Simulation results

The testbench is self-checking and covers four scenarios. All checks pass, both in Icarus Verilog and in ModelSim.

- **Test 1** - a cold read miss, then hits on the same line at different word offsets.
- **Test 2** - four distinct tags coexist in the same set (4 ways full, all hit) - impossible in a direct-mapped cache.
- **Test 3** - correct LRU: after tag0 becomes MRU, inserting tag4 evicts exactly tag1 (the least-recently-used).
- **Test 4** - write-allocate (a store on a miss brings the block in and writes the value, read back correctly) and write-back of the dirty line on eviction.